

Chủ đề:

Tìm hiểu về mã hoá SOSEMANUK

Nhóm 2:

Nguyễn Trọng Phúc - 20224099

Nguyễn Cao Quang Anh - 20223849

Phạm Đăng Bách - 20223871

Nguyễn Trương Hồng Đức - 202223919

Nguyễn Việt Dũng - 20223765

Nguyễn Thế Hào - 20223959

Trần Tuấn Tú - 20223819

ONE LOVE. ONE FUTURE

Giới thiệu

SOSEMANUK

Thành phần

Khởi tạo

Key setup

IV setup

Sinh keystream

LFSR (Linear Feedback Shift Register)

FSM (Finite State Machine)

Sinh từ keystream

Mã hoá & Giải mã

Giới thiệu

SOSEMANUK

Thành phần

Khởi tạo

Sinh keystream

Mã hoá & Giải mã

- ◇ **Định nghĩa:** Mã dòng SOSEMANUK là một mã dòng đồng bộ (synchronous stream cipher) được thiết kế chuyên biệt cho các ứng dụng phần mềm.
- ◇ **Thông số kỹ thuật:**
 - Hỗ trợ khoá có độ dài thay đổi 128-256 bit.
 - Cung cấp mức bảo mật 128 bit với mọi độ dài khóa.
- ◇ **Nguồn gốc:** Thiết kế được lấy cảm hứng từ mã dòng SNOW 2.0.
- ◇ **Thiết lập khoá:** Quy trình khởi tạo dựa trên phiên bản rút gọn của mã khối SERPENT.

Giới thiệu

SOSEMANUK

Thành phần

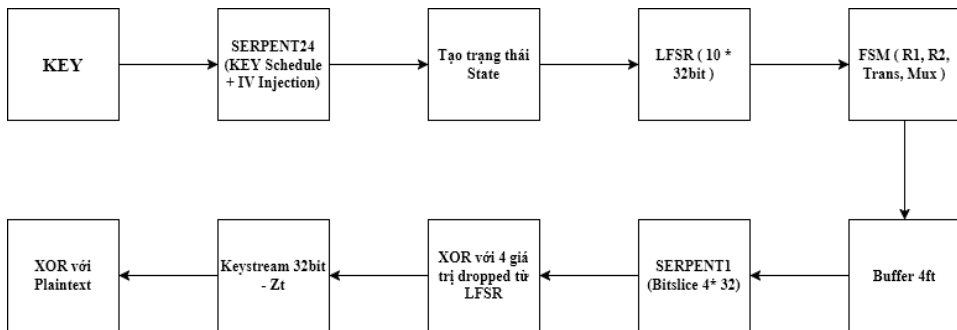
Khởi tạo

Sinh keystream

Mã hoá & Giải mã

Sosemanuk kết hợp 3 phần chính:

- ◇ **LFSR (Linear Feedback Shift Register):** Phần tuyến tính, bao gồm 10 thanh ghi 32-bit.
- ◇ **FSM (Finite State Machine):** Phần phi tuyến, bao gồm 2 thanh ghi R1, R2.
- ◇ **Serpent:** Dùng cho key/IV setup (Serpent24) và biến đổi đầu ra (Serpent1).



Giới thiệu

Khởi tạo

Key setup

IV setup

Sinh keystream

Mã hoá & Giải mã

Giới thiệu

Khởi tạo

Key setup

IV setup

Sinh keystream

Mã hoá & Giải mã

- ◇ **Mục tiêu chính:** Sử dụng thuật toán Serpent24 để sinh ra các khóa con (subkeys).
- ◇ Các khóa con này sẽ được dùng cho giai đoạn IV Setup.
- ◇ **Serpent24 là gì?**
 - Là một biến thể của mã khối SERPENT (chuẩn mã hóa 128-bit).
 - Serpent24 được rút gọn còn 24 vòng (thay vì 32 vòng như Serpent chuẩn) để tối ưu hóa hiệu năng.
- ◇ **Kết quả:** Giai đoạn này tạo ra một bộ khóa con (round keys), ký hiệu là $K_0, K_1, \dots, K_{23}$.

- ◇ Serpent24 hoạt động trên các khối dữ liệu 128-bit.
- ◇ Mỗi khối 128-bit được chia thành 4 từ (words) 32-bit, ký hiệu là (X_0, X_1, X_2, X_3) .
- ◇ Serpent24 sử dụng 24 vòng lặp thay vì 32 vòng như Serpent tiêu chuẩn.
- ◇ Mỗi vòng lặp sẽ thực hiện 3 bước chính:
 1. **XOR với khóa con:**
Thực hiện phép XOR khối dữ liệu với khóa con của vòng đó: $X_i = X_i \oplus K_i$.
 2. **Áp dụng S-box (Substitution Box - Trái tim của thuật toán):**
Sử dụng 8 S-box cố định (S0–S7) để hoán vị bit phi tuyến.
 3. **Khuếch tán bit**

Giới thiệu

Khởi tạo

Key setup

IV setup

Sinh keystream

Mã hoá & Giải mã

Mục tiêu: Tạo trạng thái ban đầu cho LFSR và FSM.

Bước 1: Mã hoá IV bằng Serpent24

- ◇ **Đầu vào:** Một khối 128-bit chính là IV.
- ◇ **Khóa:** Sử dụng các khóa con ($K_0 \dots K_{23}$) đã tạo từ bước Key Setup.
- ◇ **Quá trình:** Sau khi chạy 24 vòng Serpent24, ta lấy 3 khối output (trạng thái trung gian) tại các vòng 12, 18, và 24.

Kết quả (3 khối 128-bit)

Các output này (ký hiệu là B_i) được chia thành 4 từ 32-bit:

- ◇ $B_{12} = (b_{12,0}, b_{12,1}, b_{12,2}, b_{12,3})$
- ◇ $B_{18} = (b_{18,0}, b_{18,1}, b_{18,2}, b_{18,3})$
- ◇ $B_{24} = (b_{24,0}, b_{24,1}, b_{24,2}, b_{24,3})$

Bước 2: Gán vào LFSR & FSM

12 giá trị 32-bit thu được từ B_{12} , B_{18} , B_{24} được dùng để nạp vào 10 thanh ghi LFSR và 2 thanh ghi FSM:

Gán cho LFSR (10 thanh ghi):

- ◇ $(s_0, s_1, s_2, s_3) = B_{12}$
- ◇ $(s_4, s_5, s_6, s_7) = B_{18}$
- ◇ $(s_8, s_9) = (b_{24,0}, b_{24,1})$

Gán cho FSM (2 thanh ghi):

- ◇ $R_1 = b_{24,2}$
- ◇ $R_2 = b_{24,3}$

Bước cuối: "Warm-up"

Sau khi gán, thuật toán chạy khoảng 25 vòng "khởi động" để khuếch tán đầy đủ ảnh hưởng của Key/IV.

Giới thiệu

Khởi tạo

Sinh keystream

LFSR (Linear Feedback Shift Register)

FSM (Finite State Machine)

Sinh từ keystream

Mã hoá & Giải mã

- ◇ Sau khi khởi tạo, thuật toán bắt đầu quá trình sinh ra từng từ 32-bit của keystream.
- ◇ Quá trình này tại mỗi vòng dựa trên sự tương tác của 3 thành phần chính:
 - **LFSR (Linear Feedback Shift Register)**: Cập nhật phần trạng thái tuyến tính.
 - **FSM (Finite State Machine)**: Cập nhật phần trạng thái phi tuyến.
 - **Serpent1**: Biến đổi phi tuyến cuối cùng để tạo ra output.

Giới thiệu

Khởi tạo

Sinh keystream

LFSR (Linear Feedback Shift Register)

FSM (Finite State Machine)

Sinh từ keystream

Mã hoá & Giải mã

- ◇ LFSR là thành phần tuyến tính, chứa 10 từ (thanh ghi) 32-bit $s_0, \dots, s_9$.
- ◇ Tại mỗi vòng, các thanh ghi dịch chuyển tuyến tính:

$$(s_0, s_1, \dots, s_9) \leftarrow (s_1, s_2, \dots, s_{10})$$

Công thức cập nhật LFSR

$$s_{t+10} = (s_{t+9} \oplus (s_{t+3} \ggg 8) \oplus \text{MUL}_\alpha(s_t))$$

Trong đó:

- ◇ s_t, s_{t+3}, s_{t+9} là các thanh ghi tại vòng t .
- ◇ $\ggg 8$: Dịch vòng phải 8 bit.
- ◇ $\text{MUL}_\alpha(x)$: Phép nhân x với hằng số $\alpha = 0x54655307$ trong trường $GF(2^{32})$.

Giới thiệu

Khởi tạo

Sinh keystream

LFSR (Linear Feedback Shift Register)

FSM (Finite State Machine)

Sinh từ keystream

Mã hoá & Giải mã

- ◇ FSM là thành phần phi tuyến, mục tiêu là phá vỡ tính tuyến tính của LFSR.
- ◇ FSM có 2 thanh ghi 32-bit là R_1 và R_2 .

Công thức cập nhật FSM

Tại mỗi vòng t , FSM được cập nhật bằng 2 công thức:

1. $R1_t = (s_{t+5} + R2_t) \pmod{2^{32}}$
2. $R2_t = \text{Trans}(R1_t) \oplus s_{t+9}$

Hàm Trans(x)

Hàm Trans(x) được thiết kế để tạo khuếch tán bit mạnh, kết hợp các phép toán XOR, cộng ($\pmod{2^{32}}$) và xoay bit:

$$\text{Trans}(x) = (x \oplus (x \lll 7) \oplus (x \ggg 7)) \oplus ((x \lll 22) + (x \ggg 9))$$

Giới thiệu

Khởi tạo

Sinh keystream

LFSR (Linear Feedback Shift Register)

FSM (Finite State Machine)

Sinh từ keystream

Mã hoá & Giải mã

- ◇ Output của LFSR và FSM được kết hợp lại để tạo ra một giá trị tạm thời f_t .
- ◇ Giá trị này là đầu vào cho bộ lọc phi tuyến cuối cùng (Serpent1).

Công thức tính f_t

$$f_t = (s_{t+9} + R1_t) \pmod{2^{32}} \oplus R2_t$$

Giải thích:

- ◇ s_{t+9} : Lấy giá trị từ thanh ghi thứ 9 của LFSR.
- ◇ $R1_t$: Lấy giá trị thanh ghi R_1 của FSM (đã được cập nhật ở vòng t).
- ◇ $R2_t$: Lấy giá trị thanh ghi R_2 của FSM (đã được cập nhật ở vòng t).

- ◇ Giá trị f_t cùng 3 giá trị khác từ LFSR ($s_{t+1}, s_{t+2}, s_{t+3}$) được đưa vào hàm Serpent1.
- ◇ **Mục đích:** Áp dụng một lớp phi tuyến mạnh (S-box S2) để phá vỡ bất kỳ mối tương quan tuyến tính nào còn sót lại và tạo ra output cuối cùng z_t .

Công thức Output z_t

$$z_t = \text{Serpent1}(f_t, s_{t+1}, s_{t+2}, s_{t+3})$$

S-box S2

- ◇ Serpent1 sử dụng S-box S2 từ đặc tả của Serpent.
- ◇ S-box S2 là một bảng tra cứu (mapping) 4-bit $\rightarrow$ 4-bit.
- ◇ $S2 = [8, 6, 7, 9, 3, 12, 10, 15, 13, 1, 14, 4, 0, 11, 5, 2]$

Giới thiệu

Khởi tạo

Sinh keystream

Mã hoá & Giải mã

- ◇ SOSEMANUK là một mã dòng đồng bộ (synchronous stream cipher).
- ◇ Quá trình mã hóa và giải mã đều được thực hiện bằng phép toán $\oplus$ (XOR) byte-wise đơn giản.

Công thức Mã hóa

$$C_i = P_i \oplus Z_i$$

Công thức Giải mã

$$P_i = C_i \oplus Z_i$$

Điều kiện tiên quyết

Keystream (Z_i) ở phía gửi (mã hóa) và phía nhận (giải mã) **phải giống hệt nhau**.

Cảm ơn mọi người đã chú ý lắng nghe!

